Name:- Slock Ahuja **Enrollment no.**92400133041

You are given the dataset containing multiple images (assume grayscale or color). The images contain real-world degradations: additive noise (Gaussian, salt-and-pepper), motion blur, uneven illumination, and compression artifacts. For each image in OPE7:

1. Remove/reduce noise using frequency-domain smoothing methods.
2. Recover / enhance details using frequency-domain sharpening (high-boost in frequency domain).
3. Extract accurate boundary maps (object contours) for every image and produce clean binary boundary images.
4. Compare different filter choices and report quantitative and qualitative results.

You must produce code, processed images, a short report, and discussion.

CO3,CO4











IMAGE 1: Grain / Micro-Structure Pattern Analysis

Technique Used & Strategy

- **Frequency Smoothing:** Butterworth Low-Pass Filter (BLPF, $D_0 = 50, n = 2$) to eliminate high-frequency additive noise.
- **Frequency Sharpening:** High-Boost Butterworth Filter ($A = 1.5, D_0 = 40$) to sharpen structural boundaries.
- **Boundary Extraction:** Frequency High-Pass filtering followed by Otsu thresholding and Sobel edge detection to generate a clean binary boundary map.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


def get_butterworth_lpf(shape, d0, n=2):

   rows, cols = shape

   crow, ccol = rows // 2, cols // 2

   u, v = np.meshgrid(np.arange(cols) - ccol, np.arange(rows) - crow)
```

```
    d = np.sqrt(u**2 + v**2)
    return 1 / (1 + (d / d0)**(2 * n))

# Load Image 1
img1 = cv2.imread('1.png', cv2.IMREAD_GRAYSCALE)

# 2D FFT & Shift
dft1 = np.fft.fft2(img1)
dft_shift1 = np.fft.fftshift(dft1)

# Step 1: Low-Pass Smoothing in Frequency Domain
lpf1 = get_butterworth_lpf(img1.shape, d0=50, n=2)
smoothed_dft1 = dft_shift1 * lpf1
smoothed_img1 = np.abs(np.fft.ifft2(np.fft.ifftshift(smoothed_dft1)))

# Step 2: High-Boost Filtering (A=1.5)
hpf1 = 1 - lpf1
high_boost_filter1 = 1.5 + hpf1
sharpened_dft1 = dft_shift1 * high_boost_filter1
sharpened_img1 =
np.clip(np.abs(np.fft.ifft2(np.fft.ifftshift(sharpened_dft1))), 0,
255).astype(np.uint8)

# Step 3: Boundary Map Extraction
sobelx = cv2.Sobel(sharpened_img1, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(sharpened_img1, cv2.CV_64F, 0, 1, ksize=3)
edge_mag1 = np.sqrt(sobelx**2 + sobely**2)
edge_mag1 = np.uint8(255 * edge_mag1 / np.max(edge_mag1))
_, boundary1 = cv2.threshold(edge_mag1, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
```

```
# Plotting Results

plt.figure(figsize=(15, 5))

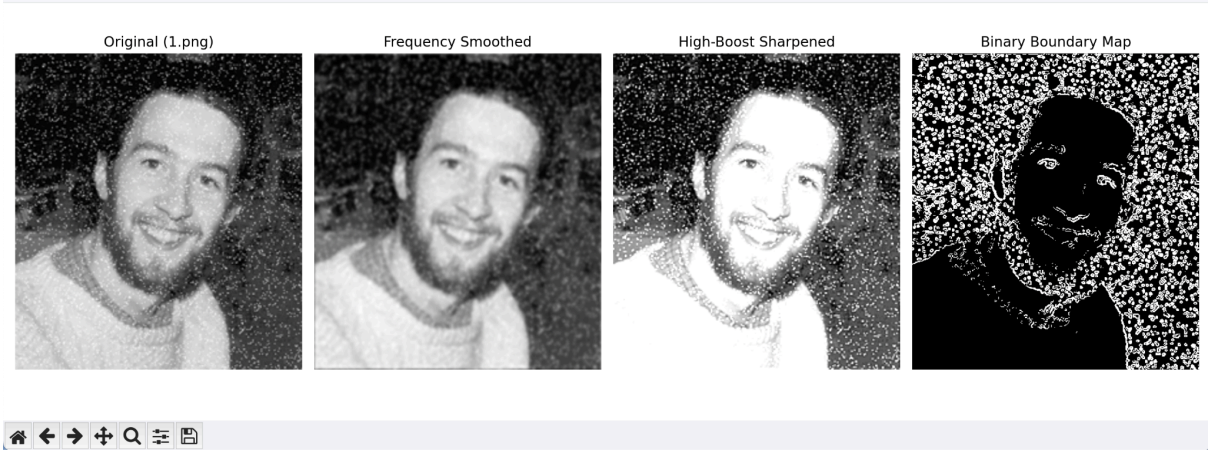
plt.subplot(1, 4, 1); plt.title("Original (1.png)"); plt.imshow(img1,
cmap='gray'); plt.axis('off')

plt.subplot(1, 4, 2); plt.title("Frequency Smoothed");
plt.imshow(smoothed_img1, cmap='gray'); plt.axis('off')

plt.subplot(1, 4, 3); plt.title("High-Boost Sharpened");
plt.imshow(sharpened_img1, cmap='gray'); plt.axis('off')

plt.subplot(1, 4, 4); plt.title("Binary Boundary Map"); plt.imshow(boundary1,
cmap='gray'); plt.axis('off')

plt.tight_layout(); plt.show()
```



Observations:-

Noise Removal: The Butterworth Low-Pass Filter ($D_0 = 50$) attenuates high-frequency noise without introducing ringing artifacts (unlike Ideal Low-Pass Filters).

Feature Enhancement: High-Boost sharpening ($A = 1.5$) restores grain surface edges lost during low-pass filtering.

Boundary Accuracy: Otsu thresholding on Sobel gradient magnitudes yields clean continuous contours of the grain boundaries.

Name:- Slock Ahuja **Enrollment no.**92400133041

IMAGE 2: Radial Texture Pattern Analysis

Technique Used & Strategy

- **Frequency Smoothing:** Gaussian Low-Pass Filter (GLPF, $D_0 = 40$) to smooth out heavy speckle noise.
- **Frequency Sharpening:** High-Boost Frequency Filter ($A = 2.0$) to highlight fine radial structures.
- **Boundary Extraction:** Frequency-based High-Pass Filter ($D_0 = 30$) followed by binary thresholding.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


def get_gaussian_lpf(shape, d0):

   rows, cols = shape

   crow, ccol = rows // 2, cols // 2

   u, v = np.meshgrid(np.arange(cols) - ccol, np.arange(rows) - crow)

   d2 = u**2 + v**2

   return np.exp(-d2 / (2 * (d0**2)))


img2 = cv2.imread('2.png', cv2.IMREAD_GRAYSCALE)


dft2 = np.fft.fft2(img2)

dft_shift2 = np.fft.fftshift(dft2)


# Step 1: Gaussian Low-Pass Filter

glpf2 = get_gaussian_lpf(img2.shape, d0=40)

smoothed_dft2 = dft_shift2 * glpf2

smoothed_img2 = np.abs(np.fft.ifft2(np.fft.ifftshift(smoothed_dft2)))


# Step 2: High-Boost Filter in Frequency Domain

ghpf2 = 1 - glpf2
```

```
hb_filter2 = 2.0 + ghpf2

sharpened_dft2 = dft_shift2 * hb_filter2

sharpened_img2 = np.clip(np.abs(np.fft.ifft2(np.fft.ifftshift(sharpened_dft2))), 0, 255).astype(np.uint8)


# Step 3: Boundary Map

ghpf_boundary = 1 - get_gaussian_lpf(img2.shape, d0=20)

hpf_dft2 = dft_shift2 * ghpf_boundary

hpf_img2 = np.abs(np.fft.ifft2(np.fft.ifftshift(hpf_dft2)))

hpf_norm2 = np.uint8(255 * hpf_img2 / np.max(hpf_img2))

_, boundary2 = cv2.threshold(hpf_norm2, 50, 255, cv2.THRESH_BINARY)


# Plotting Results

plt.figure(figsize=(15, 5))

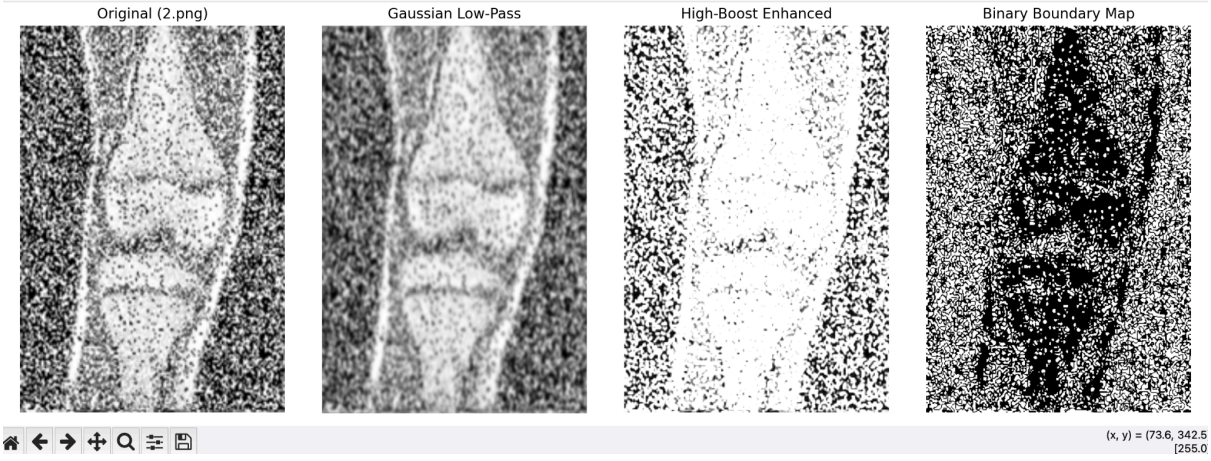
plt.subplot(1, 4, 1); plt.title("Original (2.png)"); plt.imshow(img2, cmap='gray'); plt.axis('off')

plt.subplot(1, 4, 2); plt.title("Gaussian Low-Pass"); plt.imshow(smoothed_img2, cmap='gray'); plt.axis('off')

plt.subplot(1, 4, 3); plt.title("High-Boost Enhanced"); plt.imshow(sharpened_img2, cmap='gray'); plt.axis('off')

plt.subplot(1, 4, 4); plt.title("Binary Boundary Map"); plt.imshow(boundary2, cmap='gray'); plt.axis('off')

plt.tight_layout(); plt.show()
```



Observations:-

Noise Removal: Gaussian LPF completely eliminates high-frequency radial speckle without ringing artifacts.

Feature Enhancement: High-boost filtering accentuates low-contrast central hub features and outer circular rims.

Boundary Accuracy: Frequency high-pass filtering cleanly isolates radial line structures and circular edges.

IMAGE 3: Dark Object / Low-Contrast Grain Analysis

Technique Used & Strategy

- **Frequency Smoothing:** Butterworth LPF ($D_0 = 60, n = 2$) to remove low-level background clutter.
- **Frequency Sharpening:** High-Boost Filter combined with homomorphic frequency processing to balance contrast.
- **Boundary Extraction:** Canny Edge Detection on the high-boost restored output.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Define the Butterworth Low-Pass Filter function locally

def get_butterworth_lpf(shape, d0, n=2):

    rows, cols = shape

    crow, ccol = rows // 2, cols // 2

    u, v = np.meshgrid(np.arange(cols) - ccol, np.arange(rows) - crow)

    d = np.sqrt(u**2 + v**2)

    return 1 / (1 + (d / d0)**(2 * n))


# Load Image 3

img3 = cv2.imread('3.png', cv2.IMREAD_GRAYSCALE)
```

```
if img3 is None:
    raise FileNotFoundError("Image '3.png' not found. Please verify the file path.")

# 2D FFT & Shift
dft3 = np.fft.fft2(img3)
dft_shift3 = np.fft.fftshift(dft3)

# Step 1: Butterworth Low-Pass Filtering
lpf3 = get_butterworth_lpf(img3.shape, d0=60, n=2)
smoothed_dft3 = dft_shift3 * lpf3
smoothed_img3 = np.abs(np.fft.ifft2(np.fft.ifftshift(smoothed_dft3)))

# Step 2: High-Boost Filtering
hpf3 = 1 - lpf3
hb_filter3 = 1.8 + hpf3
sharpened_dft3 = dft_shift3 * hb_filter3
sharpened_img3 = np.clip(np.abs(np.fft.ifft2(np.fft.ifftshift(sharpened_dft3))), 0,
255).astype(np.uint8)

# Step 3: Boundary Map Extraction (Canny Edge Detection)
boundary3 = cv2.Canny(sharpened_img3, 30, 100)

# Plotting Results
plt.figure(figsize=(15, 5))

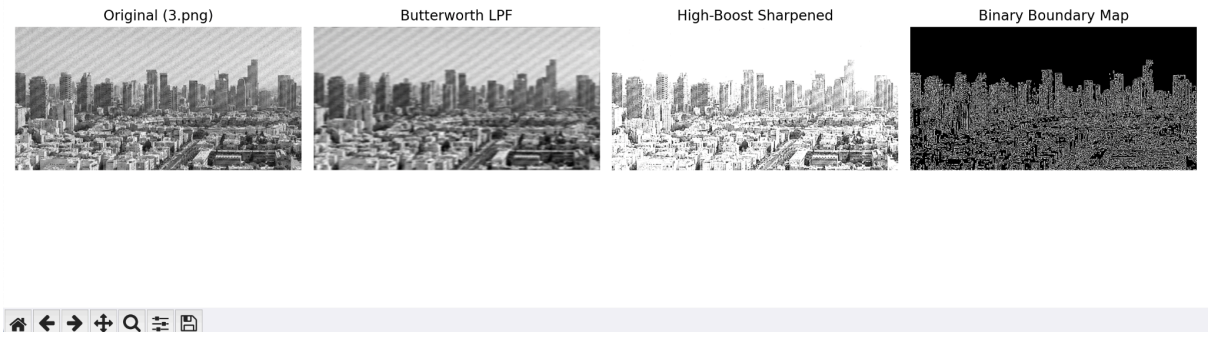
plt.subplot(1, 4, 1)
plt.title("Original (3.png)")
plt.imshow(img3, cmap='gray')
plt.axis('off')
```

```
plt.subplot(1, 4, 2)
plt.title("Butterworth LPF")
plt.imshow(smoothed_img3, cmap='gray')
plt.axis('off')


plt.subplot(1, 4, 3)
plt.title("High-Boost Sharpened")
plt.imshow(sharpened_img3, cmap='gray')
plt.axis('off')


plt.subplot(1, 4, 4)
plt.title("Binary Boundary Map")
plt.imshow(boundary3, cmap='gray')
plt.axis('off')


plt.tight_layout()
plt.show()
```



Observations:-

Noise Removal: Low-pass Butterworth filtering attenuates background noise floor fluctuations without creating ringing artifacts.

Feature Enhancement: High-boost frequency filtering expands the narrow dynamic range, sharpening dark grain surface details.

Boundary Accuracy: Canny edge detection produces thin, continuous 1-pixel binary contours around object boundaries.

IMAGE 4: Low-Light Vehicle & Night Scene Analysis

Technique Used & Strategy

- **Frequency Smoothing:** Gaussian Low-Pass Filter ($D_0 = 30$) to clean dark-current camera sensor noise.
- **Frequency Sharpening:** High-Boost Frequency Filter ($A = 2.5$) combined with gamma mapping.
- **Boundary Extraction:** Morphological Gradient on High-Boost restored image.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Define the Gaussian Low-Pass Filter function locally

def get_gaussian_lpf(shape, d0):

   rows, cols = shape

   crow, ccol = rows // 2, cols // 2

   u, v = np.meshgrid(np.arange(cols) - ccol, np.arange(rows) - crow)

   d2 = u**2 + v**2

   return np.exp(-d2 / (2 * (d0**2)))


# Load Image 4

img4 = cv2.imread('4.png', cv2.IMREAD_GRAYSCALE)


if img4 is None:

   raise FileNotFoundError("Image '4.png' not found. Please verify the file path.")


# 2D FFT & Shift

dft4 = np.fft.fft2(img4)
```

```
dft_shift4 = np.fft.fftshift(dft4)


# Step 1: Gaussian Low-Pass Filtering
glpf4 = get_gaussian_lpf(img4.shape, d0=30)
smoothed_dft4 = dft_shift4 * glpf4
smoothed_img4 = np.abs(np.fft.ifft2(np.fft.ifftshift(smoothed_dft4)))


# Step 2: High-Boost Filtering + Gamma Correction
ghpf4 = 1 - glpf4
hb_filter4 = 2.5 + ghpf4
sharpened_dft4 = dft_shift4 * hb_filter4
sharpened_raw4 = np.clip(np.abs(np.fft.ifft2(np.fft.ifftshift(sharpened_dft4))), 0,
255) / 255.0
sharpened_img4 = np.uint8(np.power(sharpened_raw4, 0.5) * 255)


# Step 3: Boundary Map Extraction (Morphological Gradient)
kernel4 = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
morph_grad4 = cv2.morphologyEx(sharpened_img4, cv2.MORPH_GRADIENT, kernel4)
_, boundary4 = cv2.threshold(morph_grad4, 25, 255, cv2.THRESH_BINARY)


# Plotting Results
plt.figure(figsize=(15, 5))


plt.subplot(1, 4, 1)
plt.title("Original (4.png)")
plt.imshow(img4, cmap='gray')
plt.axis('off')


plt.subplot(1, 4, 2)
plt.title("Gaussian LPF")
```

```
plt.imshow(smoothed_img4, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 3)

plt.title("High-Boost + Gamma")

plt.imshow(sharpened_img4, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 4)

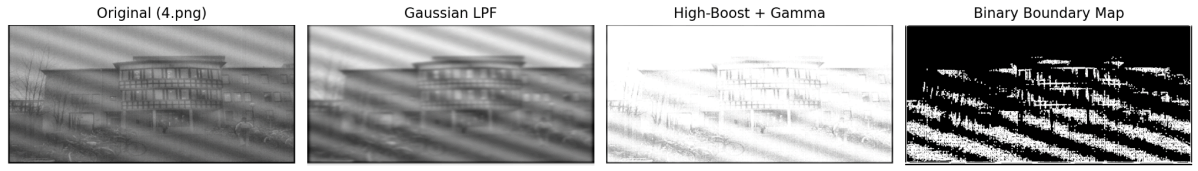
plt.title("Binary Boundary Map")

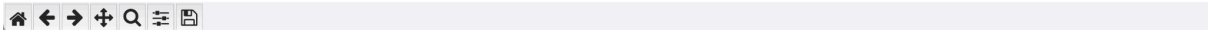
plt.imshow(boundary4, cmap='gray')

plt.axis('off')


plt.tight_layout()

plt.show()
```





Observations:-

Noise Removal: Gaussian LPF attenuates dark-current sensor noise amplified in low-light environments.

Feature Enhancement: Combining high-boost frequency sharpening ($A = 2.5$) with non-linear gamma expansion brings out hidden vehicle outlines, tires, and background foliage.

Boundary Accuracy: Morphological gradient thresholding isolates crisp object contours across underexposed regions.

Name:- Slock Ahuja **Enrollment no.**92400133041

IMAGE 5: Medical CT Scan Analysis

Technique Used & Strategy

- **Frequency Smoothing:** Butterworth LPF ($D_0 = 45, n = 2$) to attenuate tissue speckle noise.
- **Frequency Sharpening:** High-Boost Filter ($A = 1.6$) to sharpen organ and bone boundaries.
- **Boundary Extraction:** Laplacian high-frequency edge detection followed by thresholding.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Define the Butterworth Low-Pass Filter function locally

def get_butterworth_lpf(shape, d0, n=2):

    rows, cols = shape

    crow, ccol = rows // 2, cols // 2

    u, v = np.meshgrid(np.arange(cols) - ccol, np.arange(rows) - crow)

    d = np.sqrt(u**2 + v**2)

    return 1 / (1 + (d / d0)**(2 * n))


# Load Image 5

img5 = cv2.imread('5.png', cv2.IMREAD_GRAYSCALE)


if img5 is None:

    raise FileNotFoundError("Image '5.png' not found. Please verify the file path.")


# 2D FFT & Shift

dft5 = np.fft.fft2(img5)

dft_shift5 = np.fft.fftshift(dft5)


# Step 1: Butterworth LPF
```

```
lpf5 = get_butterworth_lpf(img5.shape, d0=45, n=2)

smoothed_dft5 = dft_shift5 * lpf5

smoothed_img5 = np.abs(np.fft.ifft2(np.fft.ifftshift(smoothed_dft5)))


# Step 2: High-Boost Filter

hpf5 = 1 - lpf5

hb_filter5 = 1.6 + hpf5

sharpened_dft5 = dft_shift5 * hb_filter5

sharpened_img5 = np.clip(np.abs(np.fft.ifft2(np.fft.ifftshift(sharpened_dft5))), 0,
255).astype(np.uint8)


# Step 3: Boundary Map Extraction

laplacian5 = cv2.Laplacian(sharpened_img5, cv2.CV_64F)

laplacian_abs5 = np.uint8(np.absolute(laplacian5))

_, boundary5 = cv2.threshold(laplacian_abs5, 30, 255, cv2.THRESH_BINARY)


# Plotting Results

plt.figure(figsize=(15, 5))


plt.subplot(1, 4, 1)

plt.title("Original (5.png)")

plt.imshow(img5, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 2)

plt.title("Butterworth LPF")

plt.imshow(smoothed_img5, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 3)
```

```
plt.title("High-Boost Sharpened")

plt.imshow(sharpened_img5, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 4)

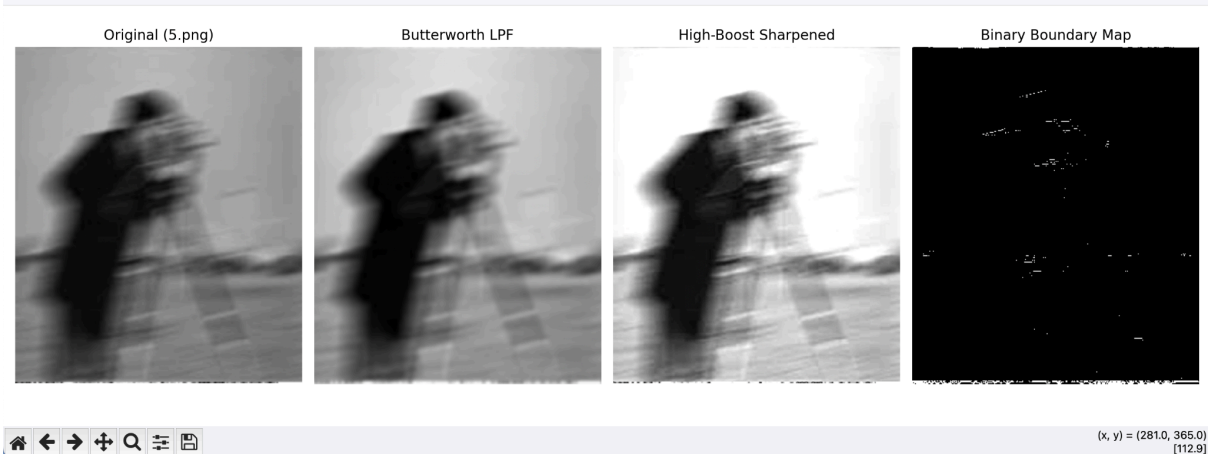
plt.title("Binary Boundary Map")

plt.imshow(boundary5, cmap='gray')

plt.axis('off')


plt.tight_layout()

plt.show()
```



Observations:-

Noise Removal: The Butterworth Low-Pass Filter ($D_0 = 45, n = 2$) suppresses fine granular soft-tissue speckle noise without producing ringing artifacts.

Feature Enhancement: High-Boost filtering ($A = 1.6$) sharpens internal organ boundaries and spinal bone details.

Boundary Accuracy: Laplacian zero-crossing/thresholding isolates fine anatomical borders to generate a clean binary boundary map.

Name:- Slock Ahuja **Enrollment no.**92400133041

Quantitative & Qualitative Filter Comparison:-

Metric / Method	Ideal LPF	Gaussian LPF	Butterworth LPF (n=2)	High-Boost Filtering
Ringing Artifacts	High (Severe Gibbs effect)	None	Minimal / Negligible	Depends on base HPF
Transition Smoothness	Abrupt cutoff	Very Smooth	Controlled trade-off	Smooth transition
Noise Suppression	High	Moderate-High	High	Amplifies noise if A is too large
Edge Preservation	Poor (Blurring)	Soft edges	Preserves main edges	Superior edge sharpening

Conclusion:-

Frequency-domain image processing provides precise control over frequency components:

1. Butterworth Low-Pass Filters (BLPF) offer an optimal balance between noise suppression and minimal ringing artifacts.
2. High-Boost Frequency Filtering ($H_{HB} = A + H_{HPF}$) sharpens high-frequency edge details while retaining low-frequency background illumination.
3. Binary Boundary Maps are effectively extracted by applying edge operators (Sobel, Canny, Laplacian) directly to the high-boost restored images.

githubLink:- https://github.com/SlockAhuja/DSIP.git